Software Language Engineering Project

Course: Seminar on Software Language Engineering (XM-0172)

February 2026

Contents

1 Introduction **2**

2 Grading **3**

3 Deliverables **5**

1 Introduction

Problem

In this assignment, you will design and implement your own Domain-Specific Language (DSL). You are free to design and implement your own executable language, but it should be non-trivial. A language with only a few language constructs, (e.g. IFTTT), is not allowed, neither are markup languages (e.g. HTML and XML).

Some examples of DSLs to inspire you:

- **Marlowe** - A language to model conditional financial agreements, payments, and expirations.
- **Alda - Music Programming Language** - A language to compose melodies using code, loops, and variables instead of a score.
- **Game Description Language** - A language to define the grid size, pieces, and movement logic for grid-based games.
- **Physical Systems Modeling** - A language to describe complex mechanical or electrical systems using mathematical equations and component connections.
- **Karel the Robot** - A language to guide a robot through a grid world using movement commands and sensor-based conditions.

This assignment applies the concepts from the course to implement a DSL from scratch. The goal is to build an executable DSL of your choice by defining its syntax, semantics, and transformation to executable code.

Understanding DSL Components

A complete DSL implementation consists of three fundamental components:

1. **Syntax Specification** - The syntax defines how programs are written and structured. The *concrete syntax* is the textual notation users write (keywords, operators, and punctuation), while the *abstract syntax* is the underlying tree structure (AST) representing the program's meaning. For example, `3 + 4` and `add(3, 4)` have different concrete syntax but could share the same abstract syntax tree.
2. **Static Semantics** - Rules that determine whether a program is well-formed beyond syntactic correctness. This includes type checking (ensuring compatible operations), scope rules (variable accessibility), name resolution (connecting uses to declarations), and domain-specific constraints. These compile-time checks catch errors before execution.

3. **Code Generation** - The process of transforming DSL programs into executable code in a target language (C, C++, Java, Python, etc.). The transformation must preserve the program's meaning. Common approaches include template-based generation, model-to-model transformation, or direct interpretation.

4. **Tooling** - The language itself is very important, but the tooling use to interact with your language is also essential. Therefore, you can think about different notations (textual, graphical, tabular, voice) or editors (e.g., notebooks, IDEs).

Language Workbench Selection

A language workbench is an integrated environment for creating programming languages, providing tools to define grammars, type systems, and code generators. We recommend using Rascal, but you may explore other workbenches such as Xtext/EMF, MPS, Spoofax, or Racket.

2 Grading

Your Tasks

1. **Implement Executable DSL**

   Create a complete implementation including syntax definitions (concrete and abstract), static semantic rules (type checking, scope analysis, validation), code generators, and build scripts. Your implementation must correctly transform DSL programs into executable code.

2. **Technical Documentation**

   Write comprehensive documentation including a language reference manual, formal grammar specification (concrete and abstract syntax), semantic rules documentation, code generation strategy, and motivation for your design choices with cited sources that influenced your decisions.

3. **Validate Your Implementation**

   Develop 5 non-trivial example programs showcasing different features of your DSL, including their generated output. Provide the generated output for each example. Include both valid programs and examples that demonstrate your error handling and validation rules.

4. **Demonstration Video**

   Produce a 5-10 minute video demonstrating: your language definition (syntax and semantics), live DSL program creation, the code generation process, and execution of generated code. Upload to YouTube/Vimeo (unlisted/private acceptable) and provide the link.

Component	**Weight**
DSL Implementation	50%
Technical Documentation	10%
Use cases and testing (Example Programs)	20%
Demonstration Video	10%
Final Presentation	10%

Table 1: Grade Distribution

3 Deliverables

You must work on a Git repository and share it with us. For submission, you must make a release in your repository and also submit it via Canvas as a single ZIP file per group named: `Group_[GROUP_NUMBER].zip` (e.g., `Group_1.zip`)

Your ZIP file must contain:

- **Source Code** — All implementation files including syntax definitions, semantic rules, interpreter/compiler, code generators, tooling, and build scripts.
- **README.md** — Setup instructions, execution guide, and URL of your demonstration video.
- **Technical Documentation** — Complete documentation in PDF format
- **Example Programs** — Minimum of 5 example DSL programs with their corresponding generated outputs and use cases.

Organise your submissions exactly as shown below:

```
Group_[GROUP_NUMBER].zip
├── src/
│   ├── syntax/
│   ├── semantics/
│   ├── codegen/
│   └── README.md
├── examples/
│   ├── example1.dsl
│   ├── example1-output.[ext]
│   ├── example2.dsl
│   ├── example2-output.[ext]
│   └── ..
└── documentation.pdf
```

Submissions that do not follow these instructions may receive a penalty.